1. Dataset & Exploratory Data Analysis: `cardiffnlp/tweet_eval` (Sentiment)

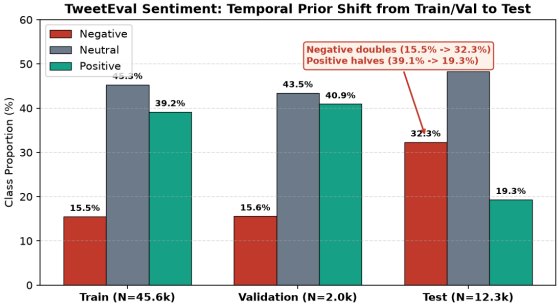
Task & Official Benchmark Splits ($N = 59,899$ Tweets)

- **Task**: 3-class Twitter sentiment ($\mathcal{Y} = \{\text{Neg } (0), \text{Neu } (1), \text{Pos } (2)\}$).
- **Challenge**: Short noisy texts (≈ 18.7 words/tweet), slang, emojis, hashtags.

Split	Tweets	Neg (0)	Neu (1)	Pos (2)
Train	45,615	7,093 (15.5%)	20,673 (45.3%)	17,849 (39.2%)
Validation	2,000	312 (15.6%)	869 (43.5%)	819 (40.9%)
Test	**12,284**	**3,972 (32.3%)**	**5,937 (48.3%)**	**2,375 (19.3%)**

Diagnostic Finding: Severe Temporal Prior Shift ($p_{\text{train}}(y) \neq p_{\text{test}}(y)$)

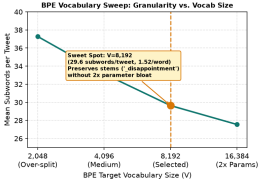
- **Prior Shift**: **Negative** doubles in Test (15.5% $\rightarrow$ 32.3%), while **Positive** halves (39.2% $\rightarrow$ 19.3%).
- **Why Both Macro-F1 & Accuracy**: Accuracy alone favors the majority class; **Macro-F1** ($\frac{1}{3}\sum_c F_{1,c}$) weights all 3 classes equally (1/3 each). We report **both** metrics across all CV and Test evaluations.
- **Class-Weighted Loss**: Standard cross-entropy under-predicts Negative; we weight the loss by $w_c = \frac{N}{3N_c}$ $(\mathbf{2.14}, \mathbf{0.74}, \mathbf{0.85})$.

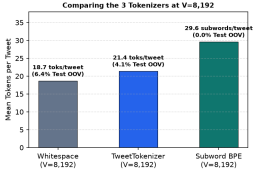


2. Text Segmentation: Whitespace vs. `TweetTokenizer` vs. Subword BPE

Why Do ALL Byte-Level BPE Vocab Sizes Have 0.0% Test OOV?

- **Word-Level Tokenizers**: **Whitespace** (6.4% Test OOV) glues punctuation (`"service!"`); **NLTK `TweetTokenizer`** (4.1% Test OOV) uses regex rules for emoticons/handles/elongation (`"cooool"` → `"coool"`).
- **Byte-Level BPE**: Starts its vocabulary with all **256 single-byte UTF-8 characters**. Any unseen word can always fall back to character pieces
  $\Longrightarrow$ **0.0% OOV at every V!**





Concrete Example: Why $V = 8,192$ Is Chosen Over 2k, 4k, 16k

Tweet: `"Unbelievable disappointment!!  Literally the worst service #fail"`

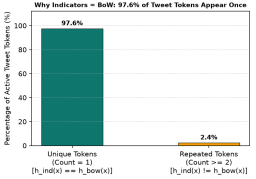
BPE Vocab V	Subword Segmentation (Toks / Tweet)	Effect on Bag-of-Words
$V = 2,048$	`['_Un','bel','iev','able','_dis','app','oint','ment',...]` (37.3)	Over-splits: `'bel'`/`'app'` collide w/ `'bell'`/`'apple'`!
$V = 4,096$	`['_Un','believ','able','_disappoint','ment',...,'f','ail']` (33.2)	Still fragments stems (`'f'` + `'ail'`).
$V = 8,192$	**`['_Un','believable','_disappointment','!!',...,'#','fail']` (29.6)**	**Sweet Spot**: Keeps full sentiment roots intact!
$V = 16,384$	`['_Unbelievable','_disappointment',...,'_#fail']` (27.6)	Memorizes rare tags; **doubles model params** ($2\times$)!

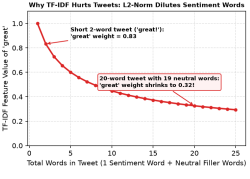
Three Unigram Feature Functions $h : \mathcal{X} \to \mathbb{R}^{8,192}$ (Lecture 2a)

1. **Token Indicators (Binary)**: $h_i^{\text{ind}}(x) = \mathbb{I}(\text{token } i \in x) \in \{0, 1\}$.
2. **Bag-of-Words (Counts in Tweet)**: $h_i^{\text{bow}}(x) = \text{tf}(i, x) \in \{0, 1, 2, \dots\}$.
3. **Unigram TF-IDF (Our Extension)**: Computed on **unigram BPE tokens**:
   - $\text{tf}(i, x)$ = token count in **that single tweet** x.
   - $\text{idf}(i) = \log\left(\frac{1+45,615}{1+\text{df}(i)}\right) + 1$ across **all** 45, 615 **training tweets**.
   - Divided by the **single tweet's** L_2 **norm**: $h_i^{\text{tfidf}}(x) = \frac{\text{tf}(i,x) \cdot \text{idf}(i)}{\|\text{tf}(x) \odot \text{idf}\|_2}$.

Concrete Vector Example: $x =$ "win win win ! the team had a great win"

Token i in Tweet	1. Indicator	2. BoW (TF)	Dataset df(i)	Dataset IDF(i)	Raw TF × IDF	3. Final TF-IDF
'the' (id=12)	1	1	18, 420 (40.4%)	1.91	1.91	0.0906
'great' (id=205)	1	1	2, 890 (6.3%)	3.76	3.76	0.1786
'win' (id=618)	**1**	**4**	**920 (2.0%)**	**4.90**	**19.61**	**0.9317**
Other 8, 185 tokens	0	0	—	—	0.00	0.0000





4. Model Architectures: Why 98%+ of MLP Parameters Sit in Layer 1

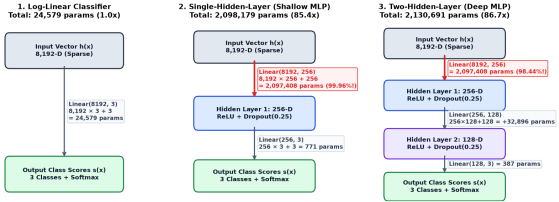
Softmax Regression Parameterisations (Lecture 2a)

Given $h(x) \in \mathbb{R}^{8,192}$, each model outputs logits $s(x) \in \mathbb{R}^3$ with $p(y = k|x) = \text{softmax}(s(x))_k$:

Layer / Connection	1. Log-Linear	2. Shallow MLP	3. Deep MLP
Layer 1 (8, 192-D Input)	$8,192 \times 3 + 3$	$\mathbf{8,192 \times 256 + 256}$	$\mathbf{8,192 \times 256 + 256}$
Parameters (% of network)	24, 579 (100%)	**2, 097, 408 (99.96%)**	**2, 097, 408 (98.44%)**
Layer 2 (Hidden / Out)	—	$256 \times 3 + 3$	$256 \times 128 + 128$
Parameters (% of network)	—	771 (0.04%)	+32, 896 (+1.54%)
Layer 3 (Output)	—	—	$128 \times 3 + 3 = 387$
Total Parameters	**24, 579 (1.0×)**	**2, 098, 179 (85.4×)**	**2, 130, 691 (86.7×)**

Why Deep MLP Barely Has More Parameters Than Shallow MLP

- **Layer 1 Bottleneck**: Projecting the 8, 192-D vocabulary to 256 units already costs **2, 097, 408** parameters (99.96% of Shallow MLP!).
- **Layer 2 (256 → 128)**: Only connects 256 units to 128 units ($256 \times 128 + 128 = \mathbf{32, 896}$ params, just +1.5% extra!).



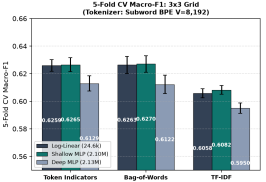
5. 5-Fold Stratified CV & Correlated Bayesian t-Test (ROPE $= [-0.01, +0.01]$)

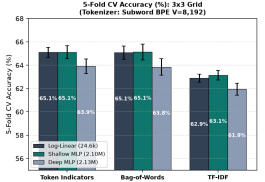
Exp 5.1: Tokenizer & BPE Vocab CV (BoW + Log-Linear)

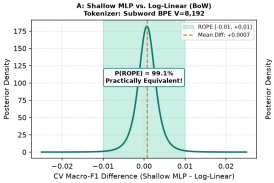
Tokenizer	CV Macro-F1	CV Accuracy	Bayesian ROPE vs. BPE-8k
1. Whitespace ($V = 8$k)	0.6091 ± 0.0051	$63.42\% \pm 0.51\%$	$P(\text{BPE-8k} > \text{WS}) = \mathbf{99.9\%}$
2. TweetTokenizer ($V = 8$k)	0.6248 ± 0.0059	$64.95\% \pm 0.54\%$	$P(\text{ROPE}) = \mathbf{100.0\%}$
3a. Subword BPE ($V = 2,048$)	0.6142 ± 0.0054	$63.95\% \pm 0.53\%$	$P(\text{BPE-8k} > 2\text{k}) = \mathbf{97.6\%}$
3b. Subword BPE ($V = 4,096$)	0.6221 ± 0.0059	$64.68\% \pm 0.55\%$	$P(\text{ROPE}) = 97.4\%$
3c. Subword BPE (V = 8, 192)	$\mathbf{0.6263 \pm 0.0062}$	$\mathbf{65.08\% \pm 0.56\%}$	**Selected Sweet Spot**
3d. Subword BPE ($V = 16,384$)	0.6258 ± 0.0061	$65.01\% \pm 0.57\%$	$P(\text{ROPE}) = \mathbf{100.0\%}$ (2× params)

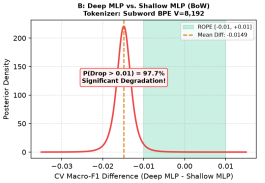
Exp 5.2 & 5.3: Full 3×3 CV Grid (Tokenizer = BPE $V = 8,192$)

Representation	Model	CV Macro-F1	CV Accuracy	Bayesian Decision (ROPE ± 0.01)
Token Indicators	Log-Linear	0.6259 ± 0.0042	$65.10\% \pm 0.43\%$	**Equivalent to BoW**: $P(\text{ROPE}) = \mathbf{98.8\%}$
Token Indicators	Shallow MLP	0.6265 ± 0.0051	$65.12\% \pm 0.55\%$	**Equivalent to BoW**: $P(\text{ROPE}) = \mathbf{99.6\%}$
Token Indicators	Deep MLP	0.6129 ± 0.0056	$63.91\% \pm 0.62\%$	**Inferior**: $P(\text{Drop} > 0.01) = \mathbf{96.1\%}$
Bag-of-Words	**Log-Linear**	$\mathbf{0.6263 \pm 0.0062}$	$\mathbf{65.08\% \pm 0.56\%}$	**Parsimonious Winner** (24.6k params)
Bag-of-Words	**Shallow MLP**	$\mathbf{0.6270 \pm 0.0060}$	$\mathbf{65.13\% \pm 0.68\%}$	**Equivalent to Log-Lin**: $P(\text{ROPE}) = \mathbf{99.1\%}$
Bag-of-Words	Deep MLP	0.6122 ± 0.0067	$63.85\% \pm 0.71\%$	**Inferior vs. Shallow**: $P(\text{Drop}) = \mathbf{97.7\%}$
TF-IDF	Log-Linear	0.6058 ± 0.0032	$62.89\% \pm 0.35\%$	**Inferior vs. BoW**: $P(\text{Drop}) = \mathbf{99.4\%}$
TF-IDF	Shallow MLP	0.6082 ± 0.0031	$63.15\% \pm 0.42\%$	**Inferior vs. BoW**: $P(\text{Drop}) = \mathbf{99.3\%}$
TF-IDF	Deep MLP	0.5950 ± 0.0039	$61.94\% \pm 0.50\%$	**Inferior**: $P(\text{Drop} > 0.01) = \mathbf{100.0\%}$









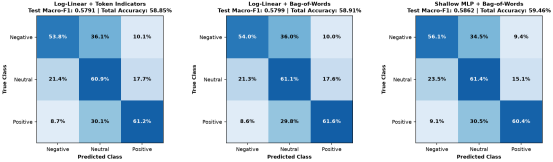
6. Official Test Set ($N = 12,284$) & Lecture 2a Error Analysis

Official Held-Out Test Set Performance (BPE $V = 8,192$)

Model	Representation	Params	CV F1	Test Macro-F1	Test Accuracy
Log-Linear	Token Indicators	24,579	0.6259	0.5791	58.85%
Log-Linear	**Bag-of-Words**	**24,579**	**0.6263**	**0.5799**	**58.91%**
Log-Linear	TF-IDF	24,579	0.6058	0.5584	56.72%
Shallow MLP	**Bag-of-Words**	**2,098,179**	**0.6270**	**0.5862**	**59.46%**
Deep MLP	Bag-of-Words	2,130,691	0.6122	0.5695	57.88%

3 Concrete Error Modes Linked to Lecture 2a Challenges

1. **Negation Scope (*Uninformative Representations*)**: `"...isn't so bad"` (True: **Neu**) → Log-Linear predicts **Pos** (`"interesting"`), Shallow MLP predicts **Neg** (`"vulnerability"`). Neither links `"isn't"` to `"bad"` without word order.
2. **Topic Bias (*Spurious Correlation*)**: Factual news with `#Palestine #Israel` (True: **Neu**) → Both predict **Neg** due to negative training prior on geopolitical hashtags.
3. **Sarcasm (*Arbitrariness of Argmax*)**: `"Wow... You always pick the right heroes"` (True: **Neg**) → Both predict **Pos** ($p > 0.78$) on literal `"Wow"`/`"heroes"`.



Key Takeaway from Confusion Matrices

All CV rankings hold on the 12,284 test tweets: **Indicators** ≈ **BoW** > **TF-IDF** and **Shallow MLP** ≈ **Log-Linear** > **Deep MLP**. Errors concentrate on the **Neutral boundary** due to unigram loss of word order and pragmatics.